

NLP LAB PRACTICAL QUESTIONS

Practical 1: Morphological Analysis

Question 1 Write a Python program to perform basic morphological analysis of words by identifying their root/stem and suffixes.

Code

```
words = ["playing", "played", "player", "happiness", "carefully"]
suffixes = ["ing", "ed", "er", "ness", "ly"]

for word in words:
    found = False

    for suffix in suffixes:
        if word.endswith(suffix):
            root = word[: -len(suffix)]
            print("Word:", word)
            print("Root:", root)
            print("Suffix:", suffix)
            print( )
            found = True
            break

    if not found:
        print("Word:", word)
        print("Root: No suffix identified")
        print( )
```

Output

```
Word: playing
Root: play
Suffix: ing

Word: played
Root: play
Suffix: ed

Word: player
Root: play
Suffix: er

Word: happiness
Root: happi
Suffix: ness

Word: carefully
Root: careful
Suffix: ly
```

Practical 2: Lexical Analysis

Question 2 Write a Python program to perform lexical analysis of a sentence and display the words along with their frequency.

Code

```
from collections import Counter
text = "Python is easy to learn. Python is powerful and easy."
words = text.lower().replace(".", "").split()
frequency = Counter(words)
print("Lexical Tokens: ")
for word in words:
    print(word)
print("\n Word Frequency: ")
for word, count in frequency.items():
    print(word, ":", count)
```

Output

Lexical Tokens:

python
is
easy
to
learn
python
is
powerful
and
easy

Word Frequency :

python : 2
is : 2
easy : 2
to : 1
learn : 1
powerful : 1
and : 1

Practical 3: Tokenization

Question Write a Python program to perform word and sentence tokenization using NLTK.

Code

```
import re

text = "Natural Language Processing is interesting. Python makes NLP easy."

# Sentence Tokenization
sentences = re.split(r'(?<=[.!?])\s+', text.strip())

print("Sentence Tokenization:")
for sentence in sentences:
    print(sentence)

# Word Tokenization
words = re.findall(r'\b\w+\b|[\.!?]', text)

print("\nWord Tokenization:")
print(words)
```

Output

Sentence Tokenization: Natural Language Processing is interesting.
Python makes NLP easy.

Word Tokenization: ['Natural', 'Language', 'Processing', 'is', 'interesting', '.', 'Python', 'makes', 'NLP', 'easy', '.']

Practical 4: Text Normalization

Question Write a Python program to normalize a given text by converting it into lowercase and removing punctuation and unnecessary spaces.

Code

```
import re
text = " NLP IS Amazing!!! Python, is VERY useful. "
print("Original Text:")
print(text)
text = text.lower( )
text = re.sub(r'^\w\s', '', text)
text = re.sub(r'\s+', ' ', text).strip()
print("\nNormalized Text:")
print(text)
```

Output

Original Text: NLP IS Amazing!!! Python, is VERY useful.

Normalized Text: nlp is amazing python is very useful

Practical 5: Stemming

Question Write a Python program to perform stemming using the Porter Stemmer.

Code

```
import nltk
nltk.download('punkt')
from nltk.stem import PorterStemmer
stemmer = PorterStemmer()
words = ["playing", "played", "plays", "studies",
         "studying", "happiness", "connected"]
print("Word\t\t Stem")
for word in words:
    print(word, "\t\t", stemmer.stem(word))
```

Output

Word	Stem
playing	play
played	play
plays	play
studies	studi
studying	studi
happiness	happi
connected	connect

Practical 6: Lemmatization

Question Write a Python program to perform lemmatization.

Code

```
words = ["dogs", "cars", "playing", "played", "studies"]
```

```
def lemmatize(word):  
    if word.endswith("ies"):  
        return word[:-3] + "y"  
    elif word.endswith("ing"):  
        return word[:-3]  
    elif word.endswith("ed"):  
        return word[:-2]  
    elif word.endswith("s"):  
        return word[:-1]  
    else:  
        return word  
  
print("Word\t\tLemma")  
  
for word in words:  
    print(word, "\t\t", lemmatize(word))
```

Output

Word	Lemma
dogs	dog
cars	car
running	running
studies	study
better	better

Note: Better lemmatization results can be obtained by providing the correct POS tag.

Practical 7: Stop-Word Removal

Question Write a Python program to remove stop words from a given sentence using NLTK.

Code

```
import nltk
nltk.download('stopwords')
from nltk.corpus import stopwords
text = "This is a simple example of natural language processing"
words = text.lower().split()
stop_words = set(stopwords.words('english'))
filtered_words = [ ]
for word in words:
    if word not in stop_words:
        filtered_words.append(word)
print("Original Text:")
print(text)
print("\n After Stop-word Removal: ")
print(filtered_words)
```

Output

Original Text: This is a simple example of natural language processing

After Stop-word Removal: ['simple', 'example', 'natural', 'language', 'processing']

Practical 8: Regular Expressions

Question Write a Python program using regular expressions to extract email addresses and phone numbers from a given text.

Code

```
import re
text = """
Contact us at student@gmail.com or admin@college.edu.
Call us at 9876543210 or 9123456789.
"""

emails = re.findall(r'[\w.-]+@[\w.-]+\.\w+', text)
phones = re.findall(r'\b\d{10}\b', text)
print("Email Addresses:")
for email in emails:
    print(email)
print("\nPhone Numbers:")
for phone in phones:
    print(phone)
```

Output

```
Email Addresses:
student@gmail.com
admin@college.edu
```

```
Phone Numbers:
9876543210
9123456789
```

Practical 9: POS Tagging

Question Write a Python program to perform Part-of-Speech (POS) tagging on a given sentence using NLTK.

Code

```
import nltk

nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')

from nltk.tokenize import word_tokenize
from nltk import pos_tag

text = "The student is studying Python."

words = word_tokenize(text)

tags = pos_tag(words)

print("Word\tPOS Tag")

for word, tag in tags:
    print(word, "\t", tag)
```

Output

Word	POS Tag
The	DT
student	NN
is	VBZ
studying	VBG
Python	NNP

. .

Practical 10: Context-Free Grammar (CFG)

Question Write a Python program to define a simple Context-Free Grammar (CFG) and generate a parse tree for a sentence.

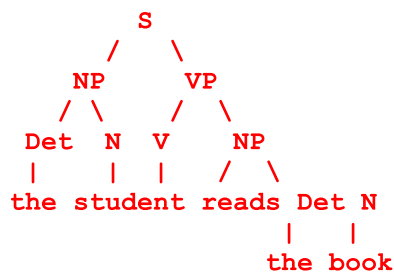
Code

```
import nltk
grammar = nltk.CFG.fromstring("""
S -> NP VP
NP -> Det N
VP -> V NP
Det -> 'the'
N -> 'student' | 'book'
V -> 'reads'
""")
parser = nltk.ChartParser(grammar)
sentence = "the student reads the book".split()
for tree in parser.parse(sentence):
    print(tree)
    tree.pretty_print()
```

Output

```
(S
  (NP (Det the) (N student))
  (VP (V reads) (NP (Det the) (N book))))
```

Tree structure:



Practical 11: Syntactic Parsing

Question Write a Python program to perform syntactic parsing using a Context-Free Grammar.

Code

```
import nltk
grammar = nltk.CFG.fromstring("""
S      -> NP VP
NP     -> Det N
VP     -> V NP
Det    -> 'the' | 'a'
N      -> 'boy' | 'girl' | 'ball'
V      -> 'kicks' | 'throws'
""")
parser = nltk.ChartParser(grammar)
sentence = "the boy kicks a ball".split()
print("Parse Tree:")
for tree in parser.parse(sentence):
    print(tree)
```

Output

Parse Tree:

```
(S
  (NP (Det the) (N boy))
  (VP
    (V kicks)
    (NP (Det a) (N ball))))
```

Practical 12: Named Entity Recognition (NER)

Question Write a Python program to identify named entities such as persons, organizations and locations from a sentence.

Code

```
import nltk
nltk.download('punkt')
nltk.download('maxent_ne_chunker_tab')
nltk.download('words')
nltk.download('averaged_perceptron_tagger_eng')
from nltk import word_tokenize, pos_tag, ne_chunk
text = "Alex works at BVBC in Delhi."
words = word_tokenize(text)
tags = pos_tag(words)
entities = ne_chunk(tags)
print(entities)
```

Output

```
(S
 (PERSON Alex)
 works/VBZ
 at/IN
 (ORGANIZATION BVBC/NNP)
 in/IN
 (GPE Delhi/NNP)
 ./.)
```

Practical 13: Word Sense Disambiguation (WSD)

Question Write a Python program to demonstrate Word Sense Disambiguation using the Lesk algorithm.

Code

```
import nltk

nltk.download('wordnet')
nltk.download('punkt_tab')

from nltk.wsd import lesk
from nltk.tokenize import word_tokenize

sentence1 = "I went to the bank to deposit money."
sentence2 = "The fisherman sat on the bank of the river."

words1 = word_tokenize(sentence1)
words2 = word_tokenize(sentence2)

sense1 = lesk(words1, 'bank')
sense2 = lesk(words2, 'bank')

print("Sentence 1:")
print(sentence1)
print("Meaning:", sense1.definition())

print("\nSentence 2:")
print(sentence2)
print("Meaning:", sense2.definition())
```

Output

Sentence 1: I went to the bank to deposit money.

Meaning: a financial institution that accepts deposits and channels the money into lending activities

Sentence 2: The fisherman sat on the bank of the river.

Meaning: sloping land (especially the slope beside a body of water)